

PROJECT REPORT - POCKET SMART AI

1. INTRODUCTION

1.1 Project Overview

PocketSmart AI is a smart financial planning and recommendation web application designed to help users manage personal expenses and plan lifestyle purchases efficiently. Built using the **Streamlit** web framework and powered by Google's **Gemini 3.8 Flash** model (**gemini-3.8-flash**) via the official **google-genai** Python SDK, PocketSmart AI bridges the gap between static expense tracking and dynamic AI-driven purchasing assistance.

- **Home Interior Planning** - Curating itemized furniture and decor packages under a set spending limit across platforms like IKEA and Amazon.
- **Event & Party Package Curations** -Generating food and accommodation bundles tailored to guest counts across Swiggy, Zomato, and OYO.
- **Curated Jewelry Suggestions** - Recommending occasion-specific traditional and modern jewelry options strictly within budget caps.

1.2 Purpose

- **Eliminate Overspending** - Deliver structured, itemized cost breakdowns that guarantee purchasing plans stay within defined financial limits.
- **Streamline Cross-Platform Research** -Eliminate manual price comparison across multiple platforms (Amazon, IKEA, Swiggy, Zomato, OYO, Flipkart) using Generative AI.
- **Integrated Financial Tracking** - Combine generative shopping advice with active monthly expense logging in a single web dashboard .

2. IDEATION PHASE

2.1 Problem Statement

Users struggle to maintain strict financial budgets due to scattered e-commerce pricing, hidden costs, and a lack of real-time itemized cost calculation tools.

2.2 Empathy Map Canvas

- **SAYS** - *I want to plan a birthday party for 10 people under ₹10,000 without accidental overspending." / "I need furniture and lighting for my study room for under ₹15,000."*
- **THINKS** - *Will buying individual decor items across multiple sites push me over budget?" / "How can I get the best value without spending hours comparing prices?"*
- **DOES** - Opens multiple browser tabs to cross-reference product prices manually; calculates running totals in physical notebooks or basic spreadsheets.
- **FEELS** - Stressed by price variations; anxious about overspending; overwhelmed by manual options.
- **GAINS** - Automated price package generation, instant category breakdown, clear visibility of remaining balance.

2.3 Brainstorming

- **Scenario-Based AI Recommendation Engine**
Tailored prompt structures feeding user constraints directly into Gemini 3.8 Flash for structured price package output.
- **Cross-Platform Price Matching**
Directing AI recommendations to reference major vendors (Amazon, IKEA, Swiggy, Zomato, OYO, Flipkart) for realistic pricing.
- **Core Expense Ledger**
Dynamic session-state financial tracker calculating real-time balance metrics.
- **Visual Analytics**
Rendered Plotly pie charts for visual spending distribution across user-defined expense categories.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

- **Setup & Authentication**
User opens Streamlit application → inputs Google Gemini API key in sidebar.
- **Tab Navigation**
The user navigates to a specific planning tab (Home Decor, Event Packages, or Jewelry).

- **Constraint Input**

The user selects preferences (e.g., Room Type, Guest Count, Occasion) and sets a maximum budget ceiling.

- **AI Output Generation**

Gemini 3.8 Flash processes inputs → generates formatted

- **Expense Logging**

User inputs daily expenses in the Core Ledger tab → metrics and Plotly analytics update instantly.

3.2 Solution Requirement

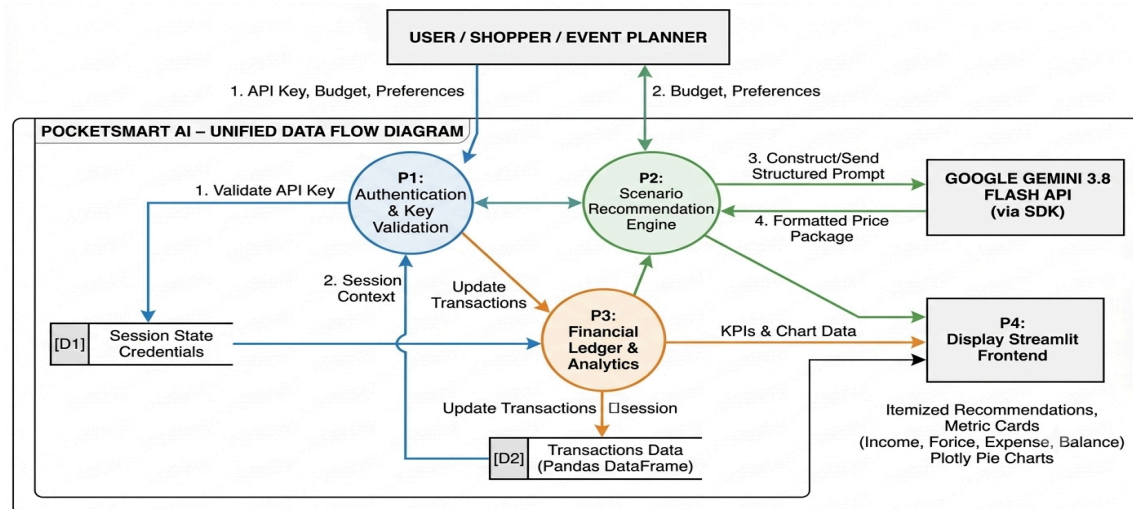
Functional Requirements:

- Secure API key entry and validation using `google-genai` client initialization.
- Input forms capturing budget limits, categories, and shopping constraints.
- Structured text generation formatted with itemized price listings and platform sources.
- Interactive data table for logging transactions (Category, Amount, Date).
- Real-time aggregate KPI metrics calculation (Total Income, Total Expenses, Remaining Balance).

Non-Functional Requirements:

- Sub-2-second generation latency using `gemini-3.8-flash`.
- Responsive, mobile-friendly Streamlit web interface.
- In-memory session state persistence during active navigation.

3.3 Data Flow Diagram



3.4 Technology Stack

- **Language:** Python 3.9+
- **Frontend / Web Framework:** Streamlit
- **Generative AI Engine:** Google Gemini 3.8 Flash (**gemini-3.8-flash**)
- **SDK:** **google-genai**
- **Data & Analytics:** Pandas
- **Data Visualization:** Plotly Express
- **Version Control & Hosting:** GitHub, Streamlit Community Cloud

4. PROJECT DESIGN

4.1 Problem Solution Fit

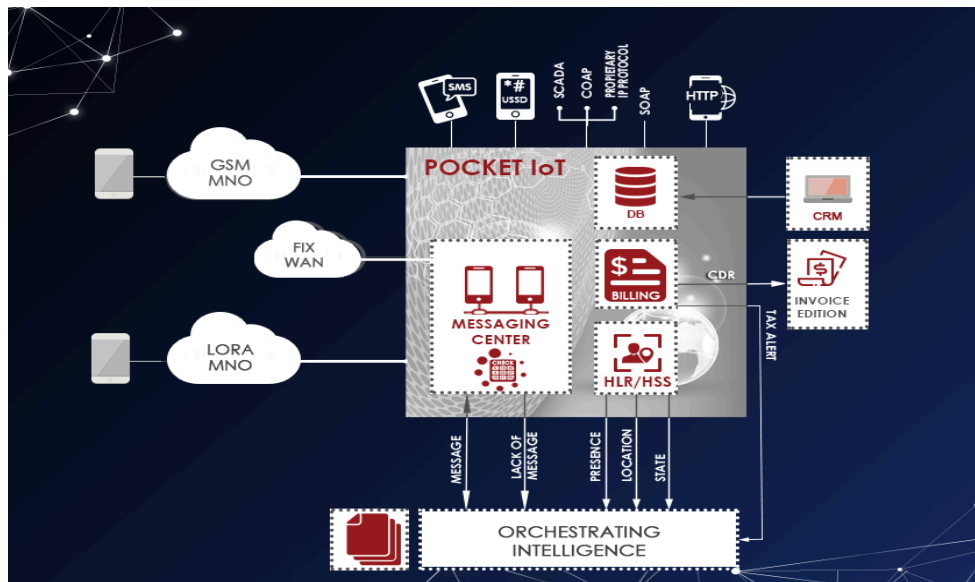
PocketSmart AI directly addresses budget anxiety by delivering pre-calculated purchase packages before spending occurs, paired with post-spending tracking to ensure overall financial health.

4.2 Proposed Solution

An AI-powered Streamlit web dashboard integrated with Google Gemini 3.8 Flash that provides itemized cross-platform recommendations (IKEA, Amazon, Swiggy, Zomato, OYO)

alongside a real-time expense ledger and visual analytics.

4.3 Solution Architecture



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase / Milestone	Tasks Completed	Primary Responsibility
Phase 1: Ideation	Problem statement definition, Empathy mapping, Brainstorming templates	Documentation Lead

Phase 2: Design & Setup	System architecture, DFD, UI layout design, GitHub repository setup	Lead Developer
Phase 3: Development	<code>app.py</code> implementation, <code>google-genai</code> SDK integration, session state ledger	Lead Developer
Phase 4: Testing	API error handling, input validation, scenario output verification	QA / Testing Lead
Phase 5: Deployment	Streamlit Cloud deployment, repository documentation completion	DevOps Lead

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

- **API Key Validation Test:** Verified system error handling when an invalid API key is provided; ensures prompt display of user feedback without application crashing.
- **Zero/Negative Budget Enforcers:** Enforced UI constraints preventing negative values or zero budget inputs.

- **Prompt Compliance Testing:** Verified that `gemini-3.8-flash` outputs itemized price totals that do not exceed user-specified budget caps across test iterations.
- **Ledger State Persistence Test:** Tested adding, editing, and deleting ledger transactions while navigating between tabs; verified data integrity in `st.session_state`.
- **Latency Verification:** Average response generation time measured under 2 seconds per scenario query.
-

7. RESULTS

7.1 Output Screenshots

- **Sidebar & Authentication:** Showing API key configuration input and Naan Mudhalvan project milestone tracking.
- **Tab 1 - Home Decor Planner:** Displaying room selection, budget slider, and generated itemized furniture/decor recommendation table.
- **Tab 2 - Party Packages:** Displaying guest count selection, event type, vendor options (Swiggy, Zomato, OYO), and AI breakdown.
- **Tab 3 - Jewelry Recommendations:** Displaying occasion selection, style preferences, and curator suggestions.
- **Tab 4 - Core Ledger Dashboard:** Displaying KPI cards (Total Income, Total Expenses, Balance), transaction table, and interactive Plotly pie chart.

8. ADVANTAGES & DISADVANTAGES

Advantages:

- **Strict Budget Adherence:** Generative AI enforces exact financial limits on recommendation sets.
- **Cross-Platform Aggregation:** Eliminates manual tab switching by combining price points across IKEA, Amazon, Swiggy, Zomato, OYO, and Flipkart.
- **Fast Execution:** Utilizing `gemini-3.8-flash` minimizes wait times for complex multi-item allocations.
- **Unified Dashboard:** Combines forward-looking purchasing

planning with backward-looking expense tracking.

Disadvantages:

- **Real-time Price Fluctuation:** AI price estimations reflect contextual knowledge; real-time vendor live stock API sync is required for precision live pricing.
- **Internet Dependency:** Requires active internet connectivity for Google Generative AI API calls.

9. CONCLUSION

PocketSmart AI successfully demonstrates the integration of Generative AI within personal financial management. By combining Google's **gemini-3.8-flash** model with Streamlit's web framework, the project delivers a user-friendly tool that mitigates financial overspending and simplifies cross-platform shopping.

10. FUTURE SCOPE

Live Price Web Scraping / API Integration: Integrating live merchant APIs (Amazon, Flipkart) for real-time stock availability and active pricing updates.

OCR Receipt Scanner: Implementing image recognition using Gemini Vision capabilities to auto-log expenses directly from uploaded store receipts.

Multi-Currency Support: Expanding financial calculations to support global currencies and regional conversion rates.

Automated Monthly Budget Alerts: Adding automated notifications when category spending approaches user-defined monthly limits.

11. APPENDIX

Source Code(if any)

Maintained in `app.py` in your GitHub repository containing the Streamlit frontend, `google-genai` integration, and Plotly visualizer.

Dataset Link

`(st.session_state)`.

GitHub & Project Demo Link

GitHub Repository:

[<https://github.com/Megha123-del-cyber/PocketSmart-AI>]

(<https://github.com/Megha123-del-cyber/PocketSmart-AI>)